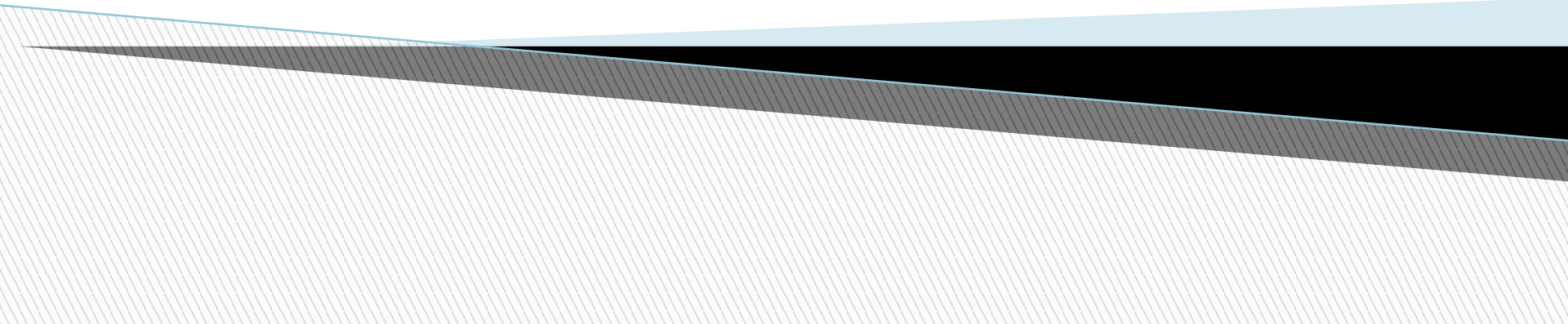


Hashing

Part (1)

Data Structure



- So far we have find, remove and insert where time varies between constant $\log_2 n$
- It would be nice to have all three as constant time operation $O(1)$.
- When the key is known, the position in the table can be accessed directly, without making any other preliminary tests, as required in a binary search or when searching a tree

Let suppose a given table

Id	Gender	Age	Adress	location
1	M	30	Model town	lahore
3	F	23	Ichra	lahore
4	M	24	Samnabad	lahore
7	F	22	Faisal Town	lahore
11	M	25	Karishan nagar	lahore
12	M	27	Chuburji	lahore
14	F	29	Cant	lahore

$O(n)$

- $O(n)$ time to find and delete of the given number of keys that are inserted consecutively.
- If this array was full and last entry was your required number then you have to traverse the N elements.

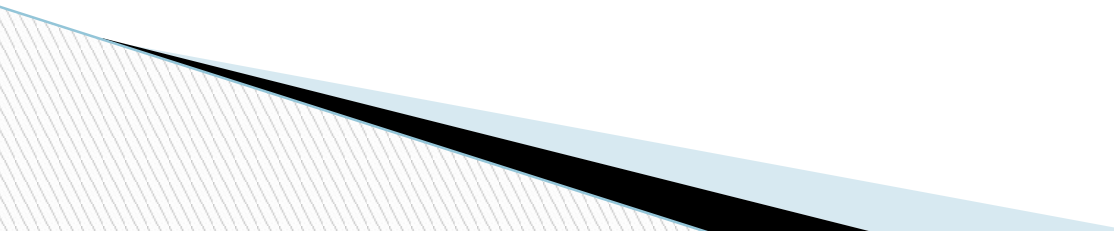
1	3	4	7	11	12	14								
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Direct indexing & Constant time $O(1)$

- We can find, insert and delete in $O(1)$ by insert taking the id as index number.
- $\text{Array}[\text{id}] = \text{id}$
- Direct indexing
 - In this technique key value is used for index number of array and key is place at that index. Example is given below.

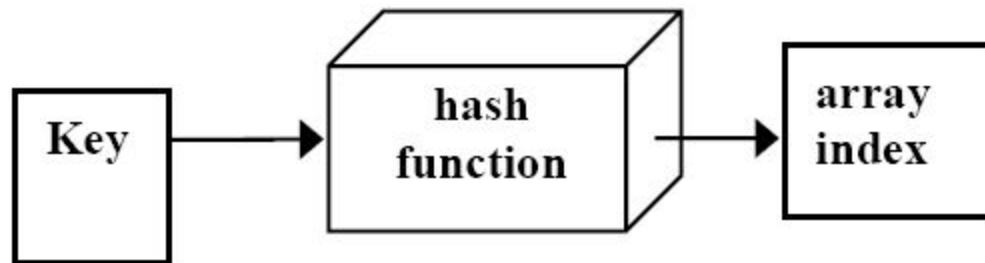
	1		3	4			7				11	12		14
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Draw backs

- Key must be integers between 0 and $N-1$
 - If the keys are not uniformly distributed, lot of space is wasted.
 - Hash function helps overcomes these.
- 

Hash function

- We will not store data in consecutive locations.
- Their place of storage is calculated using the key and a *hash function*.



O(1) with hashing

- Let suppose you have a keys from 100 to 115
- And you want to insert or find it in array in O(1) time.
- Numbers are 101,103,105,106, 107,109,113
- Here hash function is $\text{key} \% 100$ which returns the index number.
- For example $\text{Array}[\text{key} \% 100] = \text{key}$

	101		103		105	106	107		109				113	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Example other Hash Function

- Let suppose you have a keys from 100 to 300
- Here **hash function = sum of digits**.
- For 105 it is $1+0+5=6$, similarly for 103 $1+0+3=4$, for 101=2, for 107=8.
- What about $297=2+9+7=18$
- Now 18 index is greater than array size. So we have to add the mod of N in hash function.

		101		103		105		107						
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Compression function:

- Key value bigger than array size
- If hash function return the value of the index greater than the size of array then we have to take the mode of this hash function.
- For example= $\text{hash function} \% \text{size of array}$.

Hash function with compression

- Hash function $=y=$ sum of digits
- Compression $= y \bmod N$
- Here N is size of array
- For 105 it is $1+0+5=6 \bmod 15=6$
- similarly for 103 it is $1+0+3=4 \bmod 15=4$ and so on.
- If it is 297 $=2+9+7=18 \bmod 15=3$
- If it is 195 $=2+9+7=15 \bmod 15=0$

195		101	297	103		105		107						
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Hashing on string key

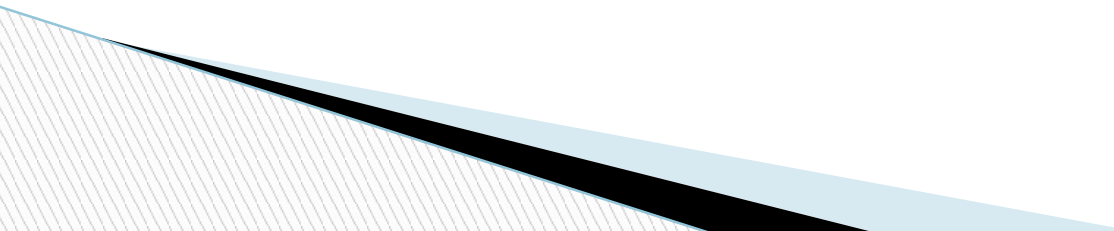
- Let suppose the key is the name of student
- The names are “kashif”, “waleed”, “tahreem”, “faham”, “Huma”, “ibrar”
- We will pass it to the hash function to get the hash key.
- Hash function
 - One possibility is to simply add the ASCII values of the characters.

$$h(str) = \left(\sum_{i=0}^{length-1} str[i] \right) \% TableSize$$

Hash Function Code c++

```
#include<string.h>
```

```
int hashCode(char* s){  
    int sum=0;  
    for(int i=0; i<strlen(s);i++){  
        sum = sum+s[i];  
    }  
    return sum % TableSize;  
}
```



Our hash function gave us the following values:

- ❑ `HashCode ("kashif") = 0` `//630%15`
- ❑ `hashCode ("waleed") = 11` `//626%15`
- ❑ `hashCode ("faham") = 14` `//509%15`
- ❑ `hashCode ("ibrar") = 3` `//528%15`
- ❑ `hashCode ("tahreem") = 7` `//742%15`
- ❑ `hashCode ("areeb") = 1` `//511%15`
- ❑ `hashCode ("ahmad") = 12` `//507%15`

0	kashif
1	areeb
2	
3	ibrar
4	
5	
6	
7	tahreem
8	
9	
10	
11	Waleed
12	ahmad
13	
14	faham

Collision

- If we want to enter a new student of name “huma” then what happened.
- $\text{hashCode}(\text{“huma”}) = 7 \quad // 427 \% 15$
- Our hash code function returns the index value of 7.
- At this index point of array tahreem is already exist.
- This problem is called collision.
- Similar case
 - $\text{abdul sami} \quad 3 \quad // 978 \% 15$
 - $\text{eisha} \quad 12 \quad // 522 \% 15$
- The definition of collision is:
 - **“When two values hash to the same array location, this is called a *collision*”**

Polynomial hash code

- ▶ Another possibility is to convert the string into some number in some arbitrary base “ a ”.

- ▶ “ a ” is positive number.

$$h(\text{str}) = [\sum_{i=0}^{\text{length}-1} \text{str}[i] * a^i] \% \text{TableSize}$$

- ▶ “ a ” *also might be a prime number*

Polynomial hash code

✚ $h(\text{str}) = [\sum_{i=0}^{\text{length}-1} \text{str}[i] * a^i] \% \text{TableSize}$

➤ $h(\text{ABC}) = (65a^0 + 66a^1 + 67a^2) \% \text{TableSize}$

➤ If TableSize=55 and a=7 then

➤ $h(\text{ABC}) = (65 * 7^0 + 66 * 7^1 + 67 * 7^2) \% 55$

➤ $h(\text{ABC}) = 15$

Our hash function result when $a=7, T=15$

- `HashCode ("kashif") =?` //Do it by yourself
- `hashCode ("waleed") =?`
- `hashCode ("faham") =?`
- `hashCode ("ibrar") =?`
- `hashCode ("tahreem") =?`
- `hashCode ("areeb") =?`
- `hashCode ("ahmad") =?`
- `hashCode ("huma") =?`
- `hashCode ("abdul sami") =?`
- `hashCode ("eisha") =?`

0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	

Result shows

- *For $a = 33, 37, 39, 41$*
- For a collection of 5000 **English** words, there were less than 7 collisions.